

Complex-Weighted Rho to ZX

Compiling weighted graph-structured lambda theories to the ZX-calculus
by symbolic execution

L. G. Meredith
F1R3FLY.io

August 12, 2026

Abstract

The quantum reading of a weighted graph-structured lambda theory (GSLT) has until now been built as the stochastic reading with $\mathbb{R}_{\geq 0}$ replaced by $\mathbb{C}$. That construction leaves phase with nowhere to live: amplitude is made to do the work of a rate, the classical marginal is recovered only by choosing a normalisation, and no interference at the level of rewriting survives. We take a different target. Given a complex-weighted GSLT cwG , we compile a program to an expression in the ZX-calculus by symbolically executing it against a cost budget. Every GSLT desugars into the rho calculus by a location-indexed installation whose location channels are injective; we use that factorisation, so only one compiler is written, from complex-weighted rho (cwrho) to ZX. Configurations are functions from locations to head tags, so the state space is a tensor product indexed by locations and the independence structure of the calculus is the tensor structure of the diagram; parallel composition is not needed and is not assumed. A rewrite instance becomes a scalar times a matrix unit on its footprint, which in ZX is a pair of disconnected spiders. One step is a block encoding: a branch register indexed by *all* jumps of the budgeted address space, a PREPARE stage whose amplitudes *are* the weight map, and a SELECT stage of controlled matrix units. The principal observation is that the stochastic and the quantum semantics are the same diagram with different caps on the branch wire: measuring the branch register in the Z basis reproduces the selection step of the stochastic simulation algorithm with rates $|w|^2$, while measuring it in the X basis — post-selecting the $|+\rangle$ outcome — performs the coherent sum over derivations. Interference is therefore effected by projection, not by unitary erasure: we prove that a retained branch record is a which-path record and suppresses interference exactly, so measurement is not a fallback for non-unitarity but the engine of coherence. We give the conditions under which a step dilates to an isometry (a per-configuration normalisation of $|w|^2$, made total by fictitious jumps at normal forms) and hence stands a chance of circuit extraction, and we record what is merely simulable otherwise. A worked example is computed, not asserted: a two-spike, two-neuron network in which two weight maps of identical magnitude are indistinguishable under the Z cap and, under the X cap, differ between never co-firing and co-firing two times in three. Spatial-behavioural key formulae compile to phase gadgets controlled on the subsystem that decides them, so the partition discipline of the weighted-GSLT note is exactly the condition that the PREPARE diagonal is single-valued, and the requirement that keys be structural is exactly the condition that the gadgets have bounded arity.

Contents

1 Introduction

3

1.1	The diagnosis	3
1.2	What changes	3
1.3	The principal observation	4
1.4	Contributions	4
1.5	What this note does not do	5
2	Complex-weighted rho	5
2.1	The desugaring, recalled	5
2.2	Weights	6
2.3	Fictitious jumps	7
3	The configuration space	7
3.1	Configurations	7
3.2	The Hilbert space	8
3.3	The budget and the address set	8
4	Rewrites as matrix units	9
4.1	Footprints	9
4.2	The elementary operator	9
5	The ZX realisation	10
5.1	Qudits to qubits	10
5.2	Matrix units are disconnected spiders	10
5.3	PREPARE: the weight map is a state	11
5.4	Key formulae are phase gadgets	12
5.5	SELECT and the one-step diagram	12
5.6	Composition to depth β	13
6	Symbolic execution: the compiler	13
6.1	The frontier	13
6.2	Reflection and drop	14
7	Three caps and what they mean	15
7.1	The Z cap: the stochastic reading	15
7.2	The X cap: the coherent reading	15
7.3	The term register: operational non-determinism	16
8	Which-path, and why measurement is the engine	17
9	Unitarity, dilation, extraction	17
9.1	Uniformisation	18
9.2	Predicate-and-permutation form	18
9.3	What is actually extractable	19
10	A worked example: a coherent spiking network	19
10.1	The theory	20
10.2	What is emitted, and how it is contracted	20
10.3	The numbers	21

10.4 Plasticity reads the branch record	22
11 Costs and tractable fragments	24
11.1 Width, depth, ancillas	24
11.2 The Clifford fragment	24
11.3 Sparsity	24
12 Implementation	25
12.1 Emission	25
12.2 The pipeline	25
12.3 What changes in the existing crate	25
12.4 Test obligations	26
13 Open threads	27
14 Conclusion	28

1 Introduction

1.1 The diagnosis

In the weighted-GSLT note [1] a finitely presentable GSLT is equipped with a weight map: rewrite rules carry weights, keys are spatial-behavioural formulae refining the left-hand side, and a map instance is part of a state specification, so that weights may change as the system runs. With weights in $\mathbb{R}_{\geq 0}$ the story is settled: the weights are propensities, redex multiplicity is Gillespie’s h , and the simulator samples a continuous-time Markov chain.

With weights in $\mathbb{C}$ the story was built by analogy, and the analogy does not hold. The difficulty is visible in the one place where the two constructions had to be reconciled: if m derivations lead to the same target, is the matrix element $\sum_i \sqrt{\lambda_i}$ or $\sqrt{\sum_i \lambda_i}$? The second was chosen, on the correct grounds that only it is a conservative lift of the classical semantics; but the choice was forced precisely because the construction had no way to keep the m derivations apart, and paying for conservativity in that coin buys a codomain in which no rewrite-level interference is possible at all. What was supposed to be the quantum theory became the classical theory with a decorative phase, and the note recorded the residue as an open thread: per-derivation amplitudes require a codomain indexed by derivations, which is a different paper.

This is that paper, and the resolution is not a better normalisation convention. It is a different target.

1.2 What changes

Three moves.

Factor through rho. Every finitely presentable GSLT desugars into the rho calculus by the location-indexed installation of [2, §4.2, App. A]. A base rewrite is installed at *every* location whose head matches, so no reduction strategy is imposed; the channel of a location is its reflection, $c(\ell) = @\ell$, which is injective; and the operational-correspondence proposition gives a step-for-step bisimulation in which each firing at ℓ is a transition labelled $c(\ell)$. Consequently the context-labelled transition labels *are* derivation identities. The codomain indexed by derivations

that the earlier note deferred is already present in the rho image, for free. We therefore write one compiler, from `cwrho` to ZX, and inherit the rest.

Make configurations the state and listeners the program. Under the desugaring a term is a parallel composition of messages $c(\ell)!(\underline{f})$, one per node, publishing that node’s head tag at that node’s location channel. The dynamic content of a running program is therefore a function from locations to head tags. This is a tensor product indexed by locations. Crucially it requires no parallel composition operator in the source: lambda, SKI and Turing machines have GSLT presentations and none of them has one, but all of them have locations, because every GSLT has one-hole contexts by construction. The tensor structure of the ZX diagram tracks the independence relation on redexes; parallel composition, where a calculus has it, merely makes that relation syntactically manifest.

Let phase live in the branch register. A single step is a sum over enabled jumps, and the ZX-calculus has composition and tensor but no primitive for linear combination. We realise the sum as a block encoding with an explicit branch register B indexed by all jumps of the budgeted address space. The weight map becomes the amplitudes of the PREPARE state on B . That is where phase lives, and it is where it does work: the branch wire is where derivations meet.

1.3 The principal observation

Once the diagram is built, the classical and the quantum readings are not two constructions but two ways of capping one wire.

Measure the branch register in the Z basis and you have the selection step of the stochastic simulation algorithm, with the jump chosen with probability proportional to $|w|^2$. Measure it in the X basis, keeping the $|+\rangle$ outcome, and you have the coherent sum over derivations, with the weights’ relative phases doing exactly what phases do.

In ZX these two are the same diagram with a green cap or a red cap on one wire. Theorems 7.1 and 7.4 make this precise. Theorem 8.1 explains why there is no third option: a retained branch record is a which-path record, and a unitary cannot erase one, so coherent recombination of distinct derivations is achieved by projection and by nothing else. Measurement is not the fallback for programs that fail to be unitary; it is the mechanism of interference.

1.4 Contributions

1. A configuration space for `cwrho` forced by the desugaring rather than chosen: a qudit per location, of dimension one more than the number of constructors (§3). Structural congruence is quotiented by construction (Proposition 3.2) and the installed listeners are invariant, hence compile to the diagram rather than to the state (Proposition 3.3).
2. Rewrite instances as scalar multiples of matrix units on footprints, with disjoint footprints tensoring (§4, Proposition 4.5).
3. The one-step block encoding $V = \sum_j |j\rangle_B \otimes K_j$ and its ZX realisation (§5), with the weight map appearing as the PREPARE amplitudes and spatial-behavioural key formulae as phase gadgets controlled on the subsystem that decides them (§5.4).

4. The symbolic execution algorithm, its termination by cost budget, and the computation of the address set (§6).
5. The two caps and their theorems: Z -basis measurement recovers the SSA selection step with rates $|w|^2$ (Theorem 7.1); X -basis post-selection performs the coherent sum (Theorem 7.4); no unitary interpolates (Theorem 8.1) (§7).
6. A dilation theorem: a uniformised weight map, made total by fictitious jumps at normal forms, yields an isometry and hence a unitary extension (§9, Theorem 9.2).
7. A worked example, computed rather than asserted (§10): a coherent spiking network whose two weight maps have identical magnitudes, hence identical stochastic behaviour, and whose coherent behaviour differs completely — co-firing probability 0 against $2/3$. And a plasticity result: a synapse-specific Hebbian read destroys exactly the coherence that produced the correlation, at the rate $\eta^2/(1+\eta^2)$ (Proposition 10.8), while a read of either index alone costs nothing (Lemma 10.7).
8. A revision of the multiplicity question. The location encoding makes m “indistinguishable” derivations genuinely distinct branches, so m and m^2 are the correct classical and coherent answers respectively, and neither is an artefact (Remark 7.6).

1.5 What this note does not do

It does not settle the elaboration of MeTTaIL tuple-pattern receives into core rho. Both source papers treat that elaboration as unnecessary detail [3, §2.2]; we work at the granularity of the context-labelled transition system, where one rule firing is one labelled transition, and record the residual obligation in §13. It does not give a qudit-native ZX treatment: we compile to qubits because the tooling does. It does not claim that any interesting program is extractable to hardware today; §9 states what would have to hold.

2 Complex-weighted rho

2.1 The desugaring, recalled

We recall only what is needed. Fix a finitely presentable GSLT G with constructor signature Σ (each $f \in \Sigma$ of arity $\text{ar}(f)$) and rewrite rules $\mathcal{R}$.

Definition 2.1 (Locations and location channels [2, Def. 4.3]). A *location* is a finite path $\ell = \langle (f_1, i_1), \dots, (f_m, i_m) \rangle$ from the root of a term, where (f, i) descends into the i -th argument of an f -node; ε is the root and $\ell \cdot (f, i)$ the child. The location channel is $c(\ell) := @\ell$. The map c is injective.

Definition 2.2 (The desugaring [2, App. A]). For a constructor f of arity n ,

$$\llbracket f(t_1, \dots, t_n) \rrbracket_\ell = c(\ell)!(\underline{f}) \mid \big|_{i=1}^n \llbracket t_i \rrbracket_{\ell \cdot (f, i)},$$

and for a base rewrite $L \Rightarrow R$ with $\text{hd}(L) = f$,

$$\llbracket L \Rightarrow R \rrbracket = \big|_{\ell: \text{hd}=f} \text{ for } (\llbracket L \rrbracket \Leftarrow c(\ell)) \{ c(\ell)!(\llbracket R \rrbracket_\ell) \mid \llbracket L \Rightarrow R \rrbracket \},$$

the inner copy re-installing the listener after each fire by the reflection idiom, with no replication. A whole GSLT is $\llbracket G \rrbracket = \big|_{(L \Rightarrow R) \in \mathcal{R}} \llbracket L \Rightarrow R \rrbracket$, with equations compiled to structural congruence. A term t is run by injecting $\llbracket t \rrbracket_\varepsilon$ on a fresh root.

Two features of this presentation do the work of this note.

Proposition 2.3 (Firings are labelled by locations). *Each base-rewrite firing of t at location ℓ corresponds to a transition of $\llbracket t \rrbracket$ labelled $c(\ell)$, and conversely; and since c is injective, distinct firings carry distinct labels.*

Proof. The correspondence is [2, Prop. 4.5]. Distinctness is injectivity of c (Definition 2.1). $\square$

Proposition 2.4 (No strategy is imposed). *The compiled program admits a firing at every location whose head matches a rule's left-hand side.*

Proof. Immediate from the indexed parallel composition $\big|_{\ell:\text{hd}=f}$ in Definition 2.2. $\square$

Proposition 2.4 is worth dwelling on, because the worked example of the companion paper [3, §5.1] compiles the lambda calculus under weak head reduction and it is easy to read the restriction as belonging to the translation. It does not: the restriction is carried by the source presentation, in the contextual rule $M \Rightarrow M' \Rightarrow \text{app}(M, N) \Rightarrow \text{app}(M', N)$, which is what encodes weak head reduction. Present lambda with contextual rules at every position and the compiled program has every β -redex live. This matters here more than it does classically: as §8 shows, the interference in a confluent calculus lives precisely in the multiplicity of derivations that a strategy would have suppressed.

2.2 Weights

Definition 2.5 (Complex-weighted GSLT). A *complex-weighted GSLT* $\text{cwG} = (G, \mathcal{K}, w)$ consists of a finitely presentable GSLT G ; for each rule $r \in \mathcal{R}$ a finite set $\mathcal{K}_r$ of *keys*, which are structural formulae of the OSLF-generated logic of G refining the left-hand side of r and forming a partition of it; and a weight map

$$w : \mathcal{C} \times \mathcal{R} \times \bigcup_r \mathcal{K}_r \longrightarrow \mathbb{C},$$

where $\mathcal{C}$ is the set of configurations (Definition 3.1). The dependence on the first argument is what makes the map dynamic — the distinguishing feature of the framework of [1] relative to SPiM — and, as §5.4 shows, it is also what makes the map a controlled gadget rather than a constant.

We carry forward two requirements from [1] without re-arguing them, and observe in §5.4 that each has an exact ZX-side meaning.

Convention 2.6 (Partition). For each rule r , the keys $\mathcal{K}_r$ partition the refinement lattice below the left-hand side of r , with a default class completing the partition.

Convention 2.7 (R3: keys are structural). Keys are drawn from the structural fragment of the generated logic — one connective per term former, plus $\wedge$, $\top$ and the complement needed for the default class. Modal keys $(\langle K \rangle, [K], \nu)$ are refused at elaboration time.

Remark 2.8 (Where the codomain restriction goes). The earlier note removed the $|z| \leq 1$ clamp and interpreted a complex weight through $\lambda(z) = |z|^2$. Both survive here, but they are no longer the whole story: $|w|^2$ is what the Z -cap sees (Theorem 7.1) and $\arg w$ is what the X -cap sees (Theorem 7.4). The interpretation map λ is the shadow the classical reading casts, not a definition of the quantum one.

2.3 Fictitious jumps

The generator convention of [1] skips self-transitions when forming the diagonal and treats them as fictitious jumps. We adopt the mirror-image convention, for a reason that becomes a theorem in §9.

Convention 2.9 (Identity jump). Every configuration carries an additional jump ι with $K_\iota = w_\iota \cdot \text{id}$, enabled exactly when no other jump is enabled. Normal forms therefore have a nonempty enabled set.

Without Convention 2.9 the block encoding of §5 annihilates normal forms, the total weight out of a normal form is zero, and the normalisation condition of Theorem 9.2 cannot hold anywhere the computation actually stops. With it, a normal form is a fixed point of the step map and the theory is total.

3 The configuration space

3.1 Configurations

Definition 3.1 (Configuration). Let Λ be a finite set of locations (the *address set*; §3.3 computes it) and $\Sigma_\perp = \Sigma \sqcup \{\perp\}$. A *configuration* is a function $\gamma : \Lambda \rightarrow \Sigma_\perp$, where $\gamma(\ell) = f$ means the message $c(\ell)!(f)$ is present and $\gamma(\ell) = \perp$ means no message is present at ℓ . Write $\mathcal{C} = \Sigma_\perp^\Lambda$.

Proposition 3.2 (Structural congruence is free). *The associativity, commutativity and unit laws of parallel composition are quotiented by Definition 3.1 without any equational machinery in the target.*

Proof. A configuration is a function on locations, not a sequence or multiset of processes. The desugaring publishes exactly one head tag per location, so the parallel composition of the messages of a term is recovered from γ up to $\equiv$, and any two $\equiv$ -equivalent message assemblies give the same γ . $\square$

This is the ZX-side counterpart of the source-side fact that equations compile to structural congruence and are “colour-respecting, cost-free, *iso* in the target rather than motion” [2, §4.2]: equations of the presented GSLT generate no diagram content whatsoever. The slogan of [1] — that a weighted GSLT is quantum exactly to the extent that its presentation withholds equations from the quotient — acquires a sharp operational form here: what is withheld from the quotient is what gets a wire.

Proposition 3.3 (Listeners are static). *The multiset of installed listeners is invariant under reduction.*

Proof. By Definition 2.2 the continuation of a firing is $c(\ell)!(\llbracket R \rrbracket_\ell) \mid \llbracket L \Rightarrow R \rrbracket$, whose second component re-installs the entire indexed family. Hence after any firing the installed family is again $\llbracket G \rrbracket$. $\square$

Proposition 3.3 is what makes the state space small. Listeners are the program; they compile to the *diagram*. Messages are the state; they compile to the *wires*. Nothing about the listener structure needs to be represented in the Hilbert space.

3.2 The Hilbert space

Definition 3.4 (State space). $\mathcal{H}_C := \bigotimes_{\ell \in \Lambda} \mathbb{C}^{\Sigma_\perp}$, with computational basis $\{|\gamma\rangle : \gamma \in \mathcal{C}\}$ and $|\gamma\rangle = \bigotimes_{\ell} |\gamma(\ell)\rangle$.

Three things are worth saying about this definition, because it is the one place where the whole design is decided.

Remark 3.5 (The tensor structure is the independence structure). The factorisation of $\mathcal{H}_C$ is indexed by locations, and locations are what every GSLT has. Two rewrite instances at locations that are not prefix-comparable act on disjoint tensor factors and commute. In a calculus with parallel composition, top-level parallel components are exactly the incomparable locations below the root, so the familiar reading of $|$ as tensor is recovered as a special case; in lambda or SKI, where disjointness of redexes is a theorem about residuals rather than a syntactic given, the same tensor decomposition is available because the same locations are.

Remark 3.6 (Contrast with $\ell^2(\text{Terms}/\equiv)$). The quantum section of [1] takes the state space to be ℓ^2 of the reachable configurations, indexed by configuration keys. That is correct and it is the right choice for a simulator that owns its own state graph. It is the wrong choice here: an unstructured index set produces a dense operator with no tensor locality, and every rewrite rule of the ZX-calculus is local. A compiler that emitted such an operator would have drawn its matrix as a diagram and gained nothing. Definition 3.4 exists to be factorised.

Remark 3.7 (Qudits). $\dim \mathbb{C}^{\Sigma_\perp} = |\Sigma| + 1$. We treat the qudit picture as primitive because it is the one the calculus dictates, and compile to qubits in §5.1 because the tooling is qubit-native. A qudit-native ZX variant would remove that compilation step.

3.3 The budget and the address set

The desugaring installs listeners at every location whose head matches, and the set of such locations is not *a priori* finite: a right-hand side may be larger than the left, and recursion, reached in rho through reflection rather than through a primitive replication, is a non-well-founded inhabitant of the process object. The risk ledger of [2] prices this exactly: a finitely presentable GSLT with finite synchronisation trees is hereditarily finite and commits no infinite risk, while recursion costs one ω . The budget is what keeps the compiler inside the finitary core.

Definition 3.8 (Budget and address set). Let $\beta \in \mathbb{N}$ be a *step budget* and t the initial term. The *address set* $\Lambda_\beta(t)$ is the set of locations occurring in any term reachable from t in at most β firings. The *jump set* $\mathcal{J}_\beta(t)$ is the set of triples $j = (r, \ell, \kappa)$ with $r \in \mathcal{R}$, $\ell \in \Lambda_\beta(t)$ a location where $\text{hd}(L_r)$ could sit, and $\kappa \in \mathcal{K}_r$, together with the identity jump ι .

We compute $\Lambda_\beta(t)$ by symbolic execution (§6); it is not assumed.

Proposition 3.9 (Width bound). $|\Lambda_\beta(t)| \leq |\Lambda(t)| \cdot g^\beta$ where g is the maximum, over rules $L \Rightarrow R$, of the number of locations of R divided by the number of locations of L consumed. Consequently $\dim \mathcal{H}_C \leq (|\Sigma| + 1)^{|\Lambda(t)| g^\beta}$.

Proof. Each firing replaces the sub-term at its location by the right-hand side, whose location count is bounded by g times that of the left; iterate β times. $\square$

The bound is honest rather than useful when $g > 1$, and it is the principal practical limit on this compiler. For calculi where a step does not grow the term — Turing machines, and lambda under a linear discipline — $g = 1$ and the width is fixed by the initial term. For β -reduction with duplication, $g > 1$ and the width grows geometrically in the budget. We return to this in §11.3.

4 Rewrites as matrix units

4.1 Footprints

Definition 4.1 (Footprint). Let $j = (r, \ell, \kappa) \in \mathcal{J}_\beta(t)$ with $r : L \Rightarrow R$. The *read set* $\text{rd}(j) \subseteq \Lambda$ is the set of locations of $\ell \cdot \text{dom}(L)$ inspected by the pattern L together with those inspected by the key κ ; the *write set* $\text{wr}(j)$ is $\ell \cdot \text{dom}(R)$ together with those locations of $\ell \cdot \text{dom}(L)$ vacated by the rewrite. The *footprint* is $\text{fp}(j) = \text{rd}(j) \cup \text{wr}(j)$.

Remark 4.2 (Why κ enters the read set). Convention 2.7 is doing work here. A structural key is a formula built from one connective per term former, so satisfaction of κ is decided by inspecting finitely many locations at bounded depth below ℓ ; the depth grading $d(\varphi)$ of [1] bounds it. A modal key would not have this property — $\langle K \rangle \varphi$ asks about the transition relation, not about the configuration — and its read set would not be a finite set of locations at all. R3 is therefore not merely a hygiene requirement inherited from a referee; it is the condition under which a footprint exists.

4.2 The elementary operator

Write $\pi_S : \mathcal{C} \rightarrow \Sigma_\perp^S$ for restriction to $S \subseteq \Lambda$, and for a pattern P at ℓ write $\hat{P}_\ell \in \Sigma_\perp^{\text{dom}}$ for the tag assignment it demands.

Definition 4.3 (Elementary operator). For $j = (r, \ell, \kappa)$ with $r : L \Rightarrow R$, and a configuration argument γ ,

$$K_j = |\hat{R}_\ell\rangle\langle\hat{L}_\ell|_{\text{fp}(j)} \otimes \text{id}_{\Lambda \setminus \text{fp}(j)},$$

a *matrix unit* on the footprint. The weight is not carried by K_j ; it is carried by the branch register (§5.3).

Note that K_j is defined for every $j \in \mathcal{J}_\beta(t)$, whether or not j is enabled at a given configuration: if γ does not match $\hat{L}_\ell$ then $K_j|\gamma\rangle = 0$. Enabledness is therefore not a side condition to be checked before building the diagram; it is the vanishing of a term in it. This is what allows the branch register to be indexed by the whole jump set, independently of the configuration — a point we return to in Remark 5.4.

Proposition 4.4 (Correctness of the elementary operator). *For $\gamma \in \mathcal{C}$, $K_j|\gamma\rangle = |\gamma'\rangle$ if the firing of r at ℓ in refinement class κ is enabled at γ and yields γ' , and $K_j|\gamma\rangle = 0$ otherwise.*

Proof. $|\hat{R}_\ell\rangle\langle\hat{L}_\ell|$ acts as the identity outside $\text{fp}(j)$ and, on $\text{fp}(j)$, sends the pattern $\hat{L}_\ell$ to $\hat{R}_\ell$ and every other assignment to 0. Matching $\hat{L}_\ell$ on $\text{rd}(j)$ is, by Definition 4.1 and Remark 4.2, exactly the conjunction of the pattern match of L and satisfaction of κ ; by Convention 2.6 exactly one

κ per (r, ℓ) can match. The resulting configuration is γ with $\text{fp}(j)$ overwritten by $\hat{R}_\ell$, which is the desugared image of the rewritten term by Definition 2.2. $\square$

Proposition 4.5 (Independence tensors). *If $\text{fp}(j) \cap \text{fp}(j') = \emptyset$ then $K_j K_{j'} = K_{j'} K_j$, and $K_j K_{j'} = (|\hat{R}\rangle\langle\hat{L}|_{\text{fp}(j)} \otimes |\hat{R}'\rangle\langle\hat{L}'|_{\text{fp}(j')}) \otimes \text{id}$.*

Proof. Operators acting on disjoint tensor factors, extended by the identity elsewhere, commute and their product is the tensor of their non-trivial parts. $\square$

Proposition 4.5 is the formal content of Remark 3.5: the independence relation on redexes is disjointness of footprints, which is decided by prefix-incomparability of locations plus the bounded depth of the patterns and keys. It is available in every GSLT, and no parallel composition operator is required to state it.

Proposition 4.6 (Rank). $\text{rank}(K_j) = (|\Sigma| + 1)^{|\Lambda| - |\text{fp}(j)|}$; in particular K_j is not unitary unless $\text{fp}(j) = \emptyset$.

Proof. A matrix unit on $\text{fp}(j)$ has rank one; tensoring with the identity on the complement multiplies the rank by the dimension of the complement. A unitary on $\mathcal{H}_C$ has full rank $(|\Sigma| + 1)^{|\Lambda|}$. $\square$

Proposition 4.6 is the reason this note takes the position it does about measurement. Non-unitarity is not an unfortunate property of badly-behaved weight maps; it is present in the very first operator we write down, for the structural reason that a rewrite destroys information about which redex fired. The ZX-calculus is indifferent — its diagrams denote arbitrary linear maps — and §9 says exactly what has to be added back to recover a unitary.

5 The ZX realisation

5.1 Qudits to qubits

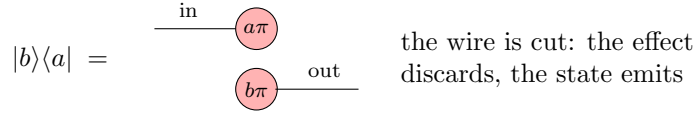
Fix $q = \lceil \log_2(|\Sigma| + 1) \rceil$ and an injection $\text{enc} : \Sigma_\perp \hookrightarrow \{0, 1\}^q$ with $\text{enc}(\perp) = 0^q$. Each location becomes a q -qubit register; $\mathcal{H}_C$ becomes $(\mathbb{C}^2)^{\otimes q|\Lambda|}$. Pattern matching becomes comparison against a constant bit string, and tag writing becomes assignment of a constant bit string, both on q qubits.

Remark 5.1. The encoding is not canonical and the choice matters for diagram size. Assigning $\text{enc}(\perp) = 0^q$ means an empty location is the all-zero register, so the vast majority of the diagram's inputs are $|0\rangle$, which ZX simplifiers exploit aggressively. A Gray-coded assignment of the remaining tags, ordering by arity, further reduces the Toffoli depth of the match predicates of §9.2. We do not optimise here.

5.2 Matrix units are disconnected spiders

Lemma 5.2 (Matrix unit). *For $a, b \in \{0, 1\}$, the map $|b\rangle\langle a|$ on one qubit is, up to a scalar, the ZX diagram consisting of two disconnected one-legged X -spiders: an effect of phase $a\pi$ on the input wire and a state of phase $b\pi$ on the output wire. For bit strings, tensor componentwise.*

Proof. The one-legged X -spider with phase 0 is $|+\rangle$ up to scalar; with phase π it is $|-\rangle$. Their Z -basis counterparts are the one-legged Z -spiders. Taking the X -spider of phase $a\pi$ as an effect gives $\langle a|$ up to scalar in the appropriate colour convention, and as a state gives $|b\rangle$; a disconnected state-effect pair denotes the outer product. $\square$



The picture makes the semantics vivid: the wire is *cut*. Everything the input said about this qubit is discarded and a constant is emitted. That is what a rewrite does to its footprint, and it is why a step is not invertible.

5.3 Prepare: the weight map is a state

Let $N = |\mathcal{J}_\beta(t)|$ and let B be a register of $b = \lceil \log_2 N \rceil$ qubits with basis $\{|j\rangle : j \in \mathcal{J}_\beta(t)\}$.

Definition 5.3 (PREPARE). For a configuration argument γ , the PREPARE state is

$$|\text{prep}(\gamma)\rangle = \sum_{j \in \mathcal{J}_\beta(t)} w(\gamma, j) |j\rangle \in \mathcal{H}_B.$$

This is the note's central relocation. In [1] the weight enters as a coefficient of a jump operator; here it enters as an amplitude of a state on a register whose basis indexes jumps. The consequences are immediate and are worth listing.

1. *Phase has somewhere to live.* $\arg w(\gamma, j)$ is a relative phase between basis vectors of $\mathcal{H}_B$, and relative phases between basis vectors of a register that is later recombined are exactly what interferes.
2. *Uniform magnitudes make PREPARE unitary.* If $|w(\gamma, j)|$ is independent of j then $|\text{prep}(\gamma)\rangle = D(\gamma)|+\rangle^{\otimes b}$ up to scalar, where $D(\gamma) = \sum_j e^{i \arg w(\gamma, j)} |j\rangle\langle j|$ is a diagonal unitary; in ZX, $|+\rangle^{\otimes b}$ is b one-legged X -spiders of phase 0, and D is a family of phase gadgets.
3. *Non-uniform magnitudes are the only source of non-unitarity in PREPARE*, and they are handled by the same post-selection that handles everything else (§5.5).
4. *The partition discipline is what makes D single-valued.* Convention 2.6 says exactly one key per (r, ℓ) matches at a given γ , so $w(\gamma, \cdot)$ is a function, so $D(\gamma)$ is a diagonal operator rather than a relation.

Remark 5.4 (Why B indexes all jumps, not the enabled ones). Indexing B by the enabled jumps at γ would make the register's dimension state-dependent, which is not a thing a wire can be. Because K_j annihilates configurations at which j is disabled (Proposition 4.4), we may index B by the whole of $\mathcal{J}_\beta(t)$ and let the disabled branches contribute zero. The price is width, $b = \lceil \log_2 N \rceil$ qubits per step; the benefit is that the diagram is configuration-independent, which is what makes it a *compiled* object rather than a trace.

5.4 Key formulae are phase gadgets

$D(\gamma)$ depends on γ , so it is not a constant diagonal but a controlled one. Its realisation is where the spatial-behavioural logic enters the diagram.

Construction 5.5 (Key gadget). Let $j = (r, \ell, \kappa)$. By Convention 2.7 and Remark 4.2, κ is a structural formula whose satisfaction at γ is a Boolean function of $\pi_{\text{rd}(j)}(\gamma)$. Compile that Boolean function to a reversible circuit MATCH_j writing a flag qubit f_j ; then apply, controlled on f_j and on $B = j$, the phase $e^{i \arg w(\gamma, j)}$, which is a Z -spider phase on the B register conjugated by the control. The magnitude $|w(\gamma, j)|$, if not uniform, is applied as a non-unitary diagonal by the same control and absorbed into the post-selection budget.

Proposition 5.6 (Gadget arity). *The gadget of Construction 5.5 touches $q \cdot |\text{rd}(j)| + b + 1$ qubits, and $|\text{rd}(j)|$ is bounded by the number of locations at depth at most $d(\kappa) + \text{depth}(L_r)$ below ℓ .*

Proof. The read set is determined by the pattern and the key; the depth grading of [1, §3] bounds the key’s reach, and the left-hand side’s depth bounds the pattern’s. $\square$

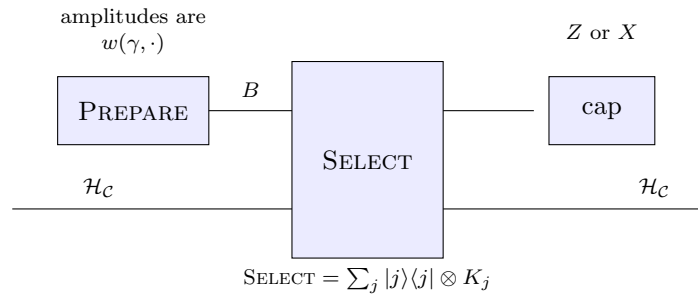
Proposition 5.6 is the payoff of R3 in its second guise. Locality of a key formula — which locations it mentions — becomes locality of the gadget — which wires it touches — which becomes tractability of simplification, since every rewrite rule of the ZX-calculus is local. A logic fragment is affordable here exactly when its formulae are decided by a bounded-arity gadget, which gives a non-arbitrary criterion for the ideal-versus-checkable split that [1] states methodologically.

5.5 Select and the one-step diagram

Definition 5.7 (SELECT). $\text{SELECT} := \sum_{j \in \mathcal{J}_\beta(t)} |j\rangle\langle j|_B \otimes K_j$, a block-diagonal operator on $\mathcal{H}_B \otimes \mathcal{H}_C$.

Definition 5.8 (One-step block encoding). $V := \text{SELECT} \circ (|\text{prep}(\cdot)\rangle \otimes \text{id}_{\mathcal{H}_C}) : \mathcal{H}_C \rightarrow \mathcal{H}_B \otimes \mathcal{H}_C$, so that

$$V|\gamma\rangle = \sum_j w(\gamma, j) |j\rangle \otimes K_j |\gamma\rangle.$$



The diagram is the whole design in one picture. The term register runs straight through. The branch register is created by PREPARE carrying the weight map, is consumed by SELECT selecting a jump, and is then capped. *Which cap* is the semantics, and §7 is about that choice.

5.6 Composition to depth β

Definition 5.9 (Budgeted diagram). $D_\beta := (\text{id}_{\mathcal{H}_B^{\otimes \beta}} \otimes \cdot)$ -composite of β copies of V , with branch registers $B_1, \dots, B_\beta$ retained:

$$D_\beta : \mathcal{H}_C \rightarrow \mathcal{H}_B^{\otimes \beta} \otimes \mathcal{H}_C, \quad D_\beta |\gamma_0\rangle = \sum_{j_1, \dots, j_\beta} \left(\prod_{i=1}^{\beta} w(\gamma_{i-1}, j_i) \right) |j_1 \cdots j_\beta\rangle \otimes |\gamma_\beta\rangle,$$

where $\gamma_i = K_{j_i} \gamma_{i-1}$ and terms with a vanishing K drop out.

Proposition 5.10 (Semigroup). $D_{\beta+1} = (\text{id}_{\mathcal{H}_B^{\otimes \beta}} \otimes V) \circ D_\beta$, and the amplitude attached to $|j_1 \cdots j_\beta\rangle \otimes |\gamma_\beta\rangle$ is the product of the weights along the unique derivation $j_1, \dots, j_\beta$.

Proof. Immediate from Definition 5.9; uniqueness of the derivation given the branch record is Proposition 2.3, since j_i determines (r, ℓ, κ) and ℓ determines the firing. $\square$

Proposition 5.10 is the correctness test we get for free: the compiler is required to emit diagrams satisfying it, and a differential harness can check $D_{\beta+1} \stackrel{?}{=} V \circ D_\beta$ by ZX-rewriting rather than by contraction.

6 Symbolic execution: the compiler

The diagram of Definition 5.9 is configuration-independent, but the jump set and the address set are not known in advance. Symbolic execution computes them.

6.1 The frontier

Definition 6.1 (Symbolic state). A *symbolic state* is a pair (Γ, k) where $\Gamma \subseteq \mathcal{C}$ is a set of configurations, represented as a set of partial tag assignments, and $k \leq \beta$ is the number of steps taken. The initial symbolic state is $(\{\gamma_0\}, 0)$ with $\gamma_0 = \pi(\llbracket t \rrbracket_\varepsilon)$.

We do *not* carry amplitudes in the symbolic state. Amplitudes are what the diagram is for; the executor's job is to discover the address set, the jump set, and the footprints, and to emit structure. This is the difference between a symbolic executor and a simulator, and it is why the executor can afford to explore all branches where the simulator of [1] samples one.

Construction 6.2 (The compiler). 1. $\Lambda \leftarrow \text{locations of } \llbracket t \rrbracket_\varepsilon$; $\Gamma \leftarrow \{\gamma_0\}$; $k \leftarrow 0$.

2. While $k < \beta$:

2.1 For each $\gamma \in \Gamma$, each rule r , each $\ell \in \Lambda$ with $\text{hd}(L_r)$ admissible at ℓ : determine the refinement class κ of γ under $\mathcal{K}_r$ (unique by Convention 2.6); record $j = (r, \ell, \kappa)$, its read set, its write set.

2.2 $\Lambda \leftarrow \Lambda \cup \bigcup_j \text{wr}(j)$.

2.3 $\Gamma \leftarrow \{K_j \gamma : \gamma \in \Gamma, j \text{ enabled at } \gamma\}$, plus γ itself for each γ with no enabled jump (the identity jump of Convention 2.9).

2.4 $k \leftarrow k + 1$.

3. $\mathcal{J}_\beta(t) \leftarrow \text{the recorded jumps}$; $\Lambda_\beta(t) \leftarrow \Lambda$.

4. Emit: the qubit layout from $\Lambda_\beta(t)$ and q ; for each step $i \leq \beta$ a fresh branch register B_i ; the PREPARE gadgets of Construction 5.5; the SELECT blocks of Definition 5.7 using Lemma 5.2; the cap chosen per §7.

Proposition 6.3 (Termination). *Construction 6.2 halts after β iterations, and $|\Lambda_\beta(t)|$ satisfies the bound of Proposition 3.9.*

Proof. The loop is bounded by β by construction; each iteration adds at most the write sets of finitely many jumps, and the growth factor is bounded as in Proposition 3.9. $\square$

Remark 6.4 (Cost accounting is the budget). We have written β as a step count for simplicity, but the intended budget is the cost account of the transpiler branch: the funding judgment funds $\Sigma \Delta := \Delta \leq \Sigma$ gates whether a redex is fundable, and an unfundable redex contributes no jump. Under that reading the identification is exact and pleasing: *the cost budget is the circuit depth*, and by Proposition 3.9 it is also, through the address set, the circuit width. The classical framework charges cost per rewrite; here the same account buys wires.

6.2 Reflection and drop

The rho calculus of the desugaring uses reflection for two things: freshness, and re-installation of listeners. Neither escapes the budget.

Freshness. Freshness is supplied by quoting, not by ν and not by a central allocator: an injected term publishes on a root $@n$ for a quoted nonce n , and locations within it are $@(n, \ell)$ [2, Rem. 4.4]. Fresh addresses are therefore computable at compile time, and a fresh location is an ancilla register initialised to $|\text{enc}(\perp)\rangle = |0\rangle^{\otimes q}$. There is no dynamic allocator to model.

Re-installation. By Proposition 3.3 the listener family is invariant, so re-installation contributes no diagram content at all.

Drop. A program that dereferences a received name, running received code, is not excluded, but its address set must be discovered rather than assumed: step 2.2 of Construction 6.2 adds the locations of whatever is dropped. Within a budget this is a finite set. The following is the boundary worth stating explicitly.

Proposition 6.5 (Reflection is Z-copy, and only on the diagonal). *Duplication of a quoted term is realised by the Z-spider, which copies computational basis states. It is therefore sound on configurations that are basis states and unsound on superpositions: there is no linear map duplicating an arbitrary $|\psi\rangle \in \mathcal{H}_C$.*

Proof. The Z-spider with one input and n outputs sends $|a\rangle \mapsto |a\rangle^{\otimes n}$ for a in the computational basis, and is linear, so it does not send a superposition to its tensor power. Non-existence of a general duplicator is the no-cloning theorem. $\square$

The consequence for the compiler is a syntactic side condition rather than a restriction on the source calculus: a jump whose right-hand side duplicates a hole-filler must be sequenced so that the duplicated register is in a basis state at the moment of duplication, or the duplication must be deferred behind a measurement. Programs in which code superposed across branches is duplicated are outside this compiler.

7 Three caps and what they mean

The one-step diagram leaves the branch register open. This section is the note's centre: the choice of cap is the choice of semantics, and the two principal choices are the two colours of spider.

7.1 The Z cap: the stochastic reading

Theorem 7.1 (Z -cap recovers the SSA selection step). *Let $\gamma \in \mathcal{C}$ be a configuration and let B be measured in the computational basis after V . Then outcome j occurs with probability*

$$\Pr[j] = \frac{|w(\gamma, j)|^2}{\sum_{j'} |w(\gamma, j')|^2 \|K_{j'}|\gamma\rangle\|^2} \|K_j|\gamma\rangle\|^2 = \frac{|w(\gamma, j)|^2 \chi_j(\gamma)}{\sum_{j'} |w(\gamma, j')|^2 \chi_{j'}(\gamma)},$$

where $\chi_j(\gamma) \in \{0, 1\}$ is enabledness, and the post-measurement state of the term register is $|K_j\gamma\rangle$. The induced process on $\mathcal{C}$ is the embedded jump chain of the weighted GSLT with rates $\lambda(w) = |w|^2$.

Proof. $V|\gamma\rangle = \sum_j w(\gamma, j)|j\rangle \otimes K_j|\gamma\rangle$, and the $|j\rangle$ are orthonormal, so the Born rule gives $\Pr[j] \propto |w(\gamma, j)|^2 \|K_j|\gamma\rangle\|^2$. By Proposition 4.4, $K_j|\gamma\rangle$ is a normalised basis vector when j is enabled and 0 otherwise, so $\|K_j|\gamma\rangle\|^2 = \chi_j(\gamma)$. Normalising gives the stated probability, which is precisely Gillespie's selection of a reaction channel with probability proportional to its propensity, with propensity $|w|^2$. $\square$

Corollary 7.2 (Multiplicity is counted). *If m distinct locations $\ell_1, \dots, \ell_m$ admit the same rule in the same refinement class with common weight w , they contribute m distinct jumps $j_1, \dots, j_m$, and the total propensity attributed to that channel is $m|w|^2$.*

Proof. The jumps are distinct because c is injective (Proposition 2.3); each contributes $|w|^2$ to the denominator of Theorem 7.1. $\square$

Corollary 7.2 is Gillespie's h , and it arrives here as a consequence of the location encoding rather than as a correction that has to be remembered. The central correctness gap identified in [1, §9.2] — that the prototype omitted multiplicity entirely — cannot be reintroduced by a compiler built this way, because there is no representation in which two redexes at different locations are the same object.

Remark 7.3 (Holding times are not in the diagram). Theorem 7.1 gives the jump chain, not the continuous-time process. The exponential holding time with parameter $a_0(\gamma) = \sum_j |w(\gamma, j)|^2 \chi_j(\gamma)$ is supplied classically, outside the diagram, exactly as the second random number of the SSA is. Note that $a_0(\gamma) = \|V|\gamma\rangle\|^2$, so the total propensity is the squared norm of the one-step image and is readable off the diagram's scalar.

7.2 The X cap: the coherent reading

Theorem 7.4 (X -cap performs the coherent sum). *Let B be measured in the X basis after V and the outcome $|+\rangle^{\otimes b}$ retained. Then, up to the scalar $2^{-b/2}$, the term register carries*

$$\sum_{j \in \mathcal{J}_\beta(t)} w(\gamma, j) K_j|\gamma\rangle,$$

Cap	Denotes	Semantics recovered
B measured in Z	$\sigma \mapsto \sum_j w_j ^2 K_j \sigma K_j^\dagger$	Gillespie / SSA jump chain
B post-selected in X	$ \psi\rangle \mapsto \sum_j w_j K_j \psi\rangle$	coherent sum over derivations
B retained	isometry V (if uniformised)	unitary dilation, no interference
term register measured	collapse to a configuration	operational non-determinism

Table 1: The four terminations of one diagram.

so derivations reaching the same configuration add with the relative phases of their weights, and the success probability of the post-selection is $\|\sum_j w(\gamma, j) K_j |\gamma\rangle\|^2 / (2^b)$ relative to the unnormalised input.

Proof. $\langle + |^{\otimes b} | j \rangle = 2^{-b/2}$ for every j , so $(\langle + |^{\otimes b} \otimes \text{id}) V |\gamma\rangle = 2^{-b/2} \sum_j w(\gamma, j) K_j |\gamma\rangle$. The success probability is the squared norm of the retained branch over the squared norm of the whole. $\square$

Corollary 7.5 (Interference is exactly diamond closure). *Two jumps $j \neq j'$ enabled at γ contribute to the same amplitude under Theorem 7.4 if and only if $K_j |\gamma\rangle = K_{j'} |\gamma\rangle$, i.e. iff the local diamond at γ closes in one step.*

Proof. The sum $\sum_j w(\gamma, j) K_j |\gamma\rangle$ groups terms by the basis vector $K_j |\gamma\rangle$; two terms share a coefficient exactly when they share that vector. $\square$

Corollary 7.5 recovers, in the compiled setting, the proposition of [1] identifying interference with closed local diamonds — but with the roles of confluence and non-confluence now clear. A confluent calculus is one all of whose diamonds close, hence one in which the X -cap has a great deal to do; the lambda calculus presented with contextual rules at every position is the interference-rich case, not the degenerate one. A deterministic calculus, such as a Turing machine, has a single derivation, so the sum has one term, no diamond closes because none opens, and the compiled diagram is a reversible computation with phases along a line. A non-confluent calculus — π , rho, the ambient calculus — has diamonds that do not close; those branches reach distinct configurations and therefore do not interfere, and what to do with them is the third cap.

7.3 The term register: operational non-determinism

Genuine non-confluence produces a superposition over distinct configurations. If the intended semantics of the source is that the program makes a non-deterministic choice — which is what the operational semantics of a mobile process calculus says — then that superposition must be resolved, and resolving it is a measurement of the term register. This is the sense in which measurement matches the program semantics rather than repairing the encoding.

Remark 7.6 (Multiplicity, revisited). Let m derivations reach a common target as in Corollary 7.2. Under the Z cap the target is reached with total probability $\propto m|w|^2$: the classical answer, linear in m . Under the X cap the amplitude is mw and the weight $m^2|w|^2$: superlinear. The earlier note identified m^2 as a normalisation artefact and repaired it by taking one square root of the summed rate rather than summing square roots. That repair was correct *for the construction it repaired*, in which the m derivations had no separate representation and the degeneracy was double-counted. Here the situation is different: the m derivations are m distinct

basis vectors of B , distinguished by injectivity of c , and the two answers are not competing normalisations of one quantity but the values of two different observables. Linear in m is what you get when you look at which derivation fired; quadratic in m is what you get when you decline to look. The m^2 is the signature of coherent recombination, and it is correct exactly when the recombination is real.

8 Which-path, and why measurement is the engine

The reader may reasonably ask why the branch register must be capped at all — why not uncompute it, coherently, and keep everything unitary. The answer is that it cannot be done, and the reason is worth stating as a theorem because it determines the architecture.

Theorem 8.1 (Which-path). *Suppose $j \neq j'$ are enabled at γ with $K_j|\gamma\rangle = K_{j'}|\gamma\rangle$. Then:*

- (i) *With B retained, the reduced state of the term register is $\sum_j |w_j|^2 K_j|\gamma\rangle\langle\gamma|K_j^\dagger$, in which the j and j' terms add incoherently: no interference occurs.*
- (ii) *There is no unitary U on $\mathcal{H}_B$ with $U|j\rangle = U|j'\rangle = |0\rangle$.*
- (iii) *Hence the only maps that recombine j and j' coherently are non-unitary: projections (equivalently, post-selected measurements).*

Proof. (i) Tracing out B from $V|\gamma\rangle\langle\gamma|V^\dagger$ kills the off-diagonal terms $|j\rangle\langle j'|$ because the $|j\rangle$ are orthonormal, leaving the stated Kraus form. (ii) A unitary is injective; $|j\rangle \neq |j'\rangle$ cannot have the same image. (iii) By (i) coherence requires the off-diagonal terms to survive, which requires B not to be traced out; by (ii) they cannot be brought to a common basis vector unitarily; a linear map sending $|j\rangle$ and $|j'\rangle$ to a common vector is non-injective, hence non-unitary, and the X -basis effect $\langle + |$ of Theorem 7.4 is such a map. $\square$

Theorem 8.1 deserves emphasis because it inverts the usual expectation. In this architecture measurement is not a concession made because programs fail to be unitary. It is the only operation that can merge derivations, and merging derivations is what interference *is*. The remark that not all programs deliver unitary matrices, and that we should be able to avail ourselves of measurement, is therefore not a caveat attached to the design; it is the design.

Corollary 8.2 (No third semantics). *Any cap on B is a linear map $\mathcal{H}_B \rightarrow \mathbb{C}$ (or a measurement thereof). The Z -basis measurements give Theorem 7.1, the $\langle + |$ effect gives Theorem 7.4, and intermediate bases interpolate: capping with $\langle \theta | = \sum_j e^{-i\theta_j} \langle j | / \sqrt{N}$ gives $\sum_j e^{-i\theta_j} w_j K_j|\gamma\rangle / \sqrt{N}$, i.e. a coherent sum with the weight map rotated by θ .*

Corollary 8.2 says the cap is not a binary switch but a dial, and that the dial is in the same space as the weight map. A partial cap — measuring some qubits of B and post-selecting others — decoheres a chosen quotient of the derivations, which is exactly the apparatus needed to say “these derivations interfere, those merely branch”. We do not develop the partial case here; it is the natural home for a treatment of decoherence induced by observation of a shard.

9 Unitarity, dilation, extraction

Everything so far denotes a linear map, which is all a ZX diagram ever needs to do, and is enough for the simulation and rewriting half of the programme. Running on hardware needs more. This section says exactly how much more.

9.1 Uniformisation

Definition 9.1 (Uniformised weight map). w is *uniformised* on a set $S \subseteq \mathcal{C}$ if for every $\gamma \in S$, $\sum_j |w(\gamma, j)|^2 \chi_j(\gamma) = 1$.

Theorem 9.2 (Dilation). *If w is uniformised on the configurations reachable within the budget, then $V : \mathcal{H}_C \rightarrow \mathcal{H}_B \otimes \mathcal{H}_C$ of Definition 5.8 is an isometry, and hence extends to a unitary $\tilde{V}$ on $\mathcal{H}_B \otimes \mathcal{H}_C$.*

Proof. $V^\dagger V = \sum_{j,j'} \overline{w_j} w_{j'} \langle j | j' \rangle K_j^\dagger K_{j'} = \sum_j |w_j|^2 K_j^\dagger K_j$. By Definition 4.3, $K_j^\dagger K_j = |\hat{L}_\ell\rangle\langle\hat{L}_\ell| \otimes \text{id}$, the projector onto configurations at which j is enabled, so $V^\dagger V |\gamma\rangle = (\sum_j |w(\gamma, j)|^2 \chi_j(\gamma)) |\gamma\rangle = |\gamma\rangle$ by hypothesis. An isometry into a space of dimension $\dim \mathcal{H}_B \cdot \dim \mathcal{H}_C > \dim \mathcal{H}_C$ extends to a unitary by completing the image's orthonormal basis. $\square$

Proposition 9.3 (Totality needs the identity jump). *Without Convention 2.9, no weight map is uniformised on a set containing a normal form.*

Proof. At a normal form no jump is enabled, so the sum in Definition 9.1 is empty and equals $0 \neq 1$. $\square$

The fictitious-jump convention of [1] therefore acquires a second, sharper justification: it was introduced so that the generator's diagonal and the SSA's total propensity would agree on self-transitions, and it turns out to be exactly what makes the quantum step total. A normal form must be a fixed point with weight of modulus one, or the machine has nothing to do and no unitary describes it.

Remark 9.4 ($[0, 1]$ versus $\mathbb{R}_{\geq 0}$, resolved). The earlier note debated whether the weight codomain should be the unit interval or $\mathbb{R}_{\geq 0}$, settled on the latter, and removed the $|z| \leq 1$ clamp. Both decisions stand. Uniformisation is a different and weaker condition: it does not bound individual weights, it constrains their sum per configuration. It is the quantum counterpart of *uniformisation* in the Markov-chain sense — converting a CTMC to a DTMC with a uniform clock — and the present theorem says that uniformisation is precisely the price of extractability. Weight maps that are not uniformised are perfectly good diagrams; they are simply not circuits.

9.2 Predicate-and-permutation form

Theorem 9.2 guarantees a unitary extension abstractly. To hand PyZX something it can extract, the extension must be realised. The following decomposition does it, at the price of a syntactic condition on the rule set.

Construction 9.5 (Predicate and permutation). Write $K_j = U_j P_j$ where $P_j = |\hat{L}_\ell\rangle\langle\hat{L}_\ell| \otimes \text{id}$ is the enabledness projector and U_j is any permutation of $\Sigma_\perp^{\text{fp}(j)}$ sending $\hat{L}_\ell$ to $\hat{R}_\ell$, extended by the identity. Then:

1. MATCH_j : a reversible circuit computing χ_j into a flag f_j , from $\pi_{\text{rd}(j)}$. Comparison against a constant bit string; Toffoli depth $O(q |\text{rd}(j)|)$.
2. WRITE_j : U_j controlled on f_j and on $B = j$. A controlled permutation, hence a reversible circuit.
3. MATCH_j^{-1} : uncompute f_j by testing whether the footprint now matches $\hat{R}_\ell$.

Definition 9.6 (Write-detectability). A rule set is *write-detectable* if for every j , and every configuration γ , the footprint of j matches $\hat{R}_\ell$ after WRITE_j if and only if it matched $\hat{L}_\ell$ before. Equivalently, no configuration matches both $\hat{L}_\ell$ and $\hat{R}_\ell$ at ℓ .

Proposition 9.7 (Flag uncomputation). *If the rule set is write-detectable, step 3 of Construction 9.5 returns f_j to $|0\rangle$, and the composite $\text{MATCH}_j^{-1}\text{WRITE}_j\text{MATCH}_j$ is a unitary on $\mathcal{H}_B \otimes \mathcal{H}_C$ agreeing with $|j\rangle\langle j| \otimes K_j$ on the enabled subspace.*

Proof. Write-detectability makes the post-write test the same Boolean function of the post-write state as the pre-write test was of the pre-write state, so the second MATCH XORs the same bit into f_j and clears it. Each of the three factors is reversible, so the composite is. $\square$

If a rule set is not write-detectable — a rule whose right-hand side can look like its own left-hand side, of which the identity jump is the extreme case — the flag cannot be uncomputed and is garbage. Garbage is discarded, which decoheres, which returns us to Theorem 8.1: the failure of write-detectability is precisely a failure to be able to tell which derivation ran, and the diagram registers it as such. The identity jump is exempt because it is a fixed point and needs no flag.

9.3 What is actually extractable

Stacking the conditions gives a three-tier statement, which we record as the honest answer to “when can this run on hardware”.

1. *Always.* A cwG program plus a budget compiles to a ZX diagram denoting a linear map. No conditions. This is the tier at which PyZX rewriting and classical simulation by contraction apply.
2. *Under uniformisation.* V is an isometry and dilates (Theorem 9.2); under write-detectability the dilation is realised by Construction 9.5 as a reversible circuit plus phase gadgets.
3. *Under extraction.* The simplified diagram must additionally admit a gflow (or another correctness condition) for PyZX’s circuit extraction to succeed. Simplification does not preserve circuit-likeness, so this is checked after simplification, not before. Where it fails, the diagram remains a measurement pattern with post-selection, and post-selection costs sampling overhead exponential in the number of post-selected qubits.

Tier 3 is why hardware is a later back end rather than a near-term target, and the note does not claim otherwise. The immediate value is tiers 1 and 2: a compiled, simplifiable, contractible object with a proof obligation attached to each of its two possible caps.

10 A worked example: a coherent spiking network

The weighted-GSLT note carried a spiking network as its worked example, with synaptic weights living in the weight map rather than in the payload and plasticity as the map’s update function. We keep that skeleton and change the codomain to $\mathbb{C}$. The question this section answers is what the phase buys, and the answer is that it correlates neurons.

Everything below is contracted out of an emitted ZX diagram. Section 10.2 says how; §10.3 gives the numbers, which are cross-checked against an independently written implementation of Definition 5.9 to a worst deviation of 8.9×10^{-16} .

10.1 The theory

Take a one-constructor signature, so $|\Sigma| + 1 = 2$ and $q = 1$: a location either carries a spike or does not. Locations are two release sites s_1, s_2 and two somas a, b . For $x \in \{1, 2\}$ and $y \in \{a, b\}$ there are two rules,

$$\text{relay}(x, y) : s_x=1, y=0 \Rightarrow s_x=0, y=1, \quad \text{leak}(x, y) : s_x=1, y=1 \Rightarrow s_x=0, y=1.$$

Transmission moves a spike from a release site to a silent soma; a spike arriving at a soma that is already spiking is lost. The leak rule is not decoration: without it the only reachable configuration after two firings is $(a=1, b=1)$ and there is nothing to correlate. It is also the repair the earlier note arrived at independently, where a hard capacity made a saturating network absorbing rather than live.

The jump index is $j = (r, x, y)$ with $r \in \{\text{relay}, \text{leak}\}$, so $|\mathcal{J}| = 8$ and the branch register is three qubits per step. The weight map is

$$w(\text{relay}, x, y) = t_{xy}, \quad w(\text{leak}, x, y) = \lambda,$$

with t a 2×2 matrix of synaptic amplitudes. The budget is $\beta = 2$: exactly enough to consume both spikes.

Proposition 10.1 (The co-firing configuration is a closed diamond). *The configuration $(a=1, b=1)$ is reached from $(s_1=1, s_2=1, a=0, b=0)$ by exactly four derivations of length two — two pairings of spikes to somas, each in two orders — with amplitudes $t_{1a}t_{2b}$, $t_{1b}t_{2a}$, $t_{2b}t_{1a}$, $t_{2a}t_{1b}$. The configurations $(1, 0)$ and $(0, 1)$ are reached by two derivations each, with amplitudes λt_{1y} and λt_{2y} . The configuration $(0, 0)$ is unreachable, since leak is disabled at a silent soma.*

Proof. Direct enumeration of the two-step derivations, using Proposition 4.4 for enabledness. $\square$

By Corollary 7.5 the four co-firing amplitudes interfere under the X cap and do not under the Z cap. That is the whole example.

10.2 What is emitted, and how it is contracted

The emitter follows §5 without shortcuts. PREPARE is realised by the multiplexed-rotation state preparation of a general n -qubit amplitude vector, so the weight map really is a state on the branch register and not a post-processing step. SELECT is realised in the predicate-and-permutation form of Construction 9.5: a reversible MATCH writes $\chi_{j(B)}(\gamma)$ into a flag from the read set, the flag is left in place, WRITE applies the controlled permutation, and the flag is capped with a $|1\rangle$ effect. That effect is the projection: branch records whose selected jump is disabled are annihilated rather than acting as the identity, which is the distinction Construction 9.5 otherwise leaves open. Of the 64 two-step branch records, 8 survive it.

The emitted diagram for the two-step network has 1806 vertices and 2244 edges. Two engineering findings are worth recording because they bear on §12's recommendation.

Remark 10.2 (Contraction order, not diagram size, is the bottleneck). A contraction that walks the vertices in order is unusable here: on the reduced 118-vertex diagram it demands a 64 GiB intermediate. Emitting each spider at bond dimension two — a spider of degree d is $\sum_k c_k |k\rangle^{\otimes d}$, hence a chain of rank-three copy tensors rather than a dense rank- d array — and choosing the path by a standard optimiser contracts the same diagram in seconds. Any implementation of §12 should provide its own contractor and not rely on the reference one.

Remark 10.3 (Simplification can make contraction harder). Full simplification takes the network diagram from 1806 vertices to 118, and makes it *harder* to contract: the reduced diagram exceeds memory where the unreduced one contracts in about three seconds. Simplification trades vertex count for connectivity, and contraction cost is governed by treewidth. This cuts against the natural assumption that simplification is always the first move; on the minimal diagram of §10.3 it is an unambiguous win (10 vertices to 2, tensor preserved), and on the network diagram it is not.

10.3 The numbers

Convergent input, and inhibition as a phase. First the minimal closed diamond: one release site, two rules with identical left- and right-hand sides — two synaptic routes to one soma — so $K_{j_1} = K_{j_2}$ and the diamond closes in a single step. Contracting the one-step diagram of Definition 5.8 under each cap:

	relative phase	Z cap: $\sum_j w_j ^2$	X cap: $ \sum_j w_j ^2$
excitatory	0	1.000000	2.000000
inhibitory	π	1.000000	0.000000
quadrature	$\pi/2$	1.000000	1.000000
one route	—	1.000000	1.000000

Remark 10.4 (Inhibition needs no negative rate). The classical framework has weights in $\mathbb{R}_{\geq 0}$, so inhibition must be encoded structurally — a capacity guard, a competing consumer, a leak. Here it is a relative phase of π on a synapse and it cancels exactly, by the same mechanism that produces excitation. This is the smallest thing the complex codomain buys that the real one cannot express at all.

Two spikes, two neurons. Now the network of §10.1, with $\lambda = 1/\sqrt{2}$ and two weight maps of *identical magnitudes*: a beamsplitter $t = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ i & -i \end{pmatrix}$ and its phase-free counterpart $t = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$.

outcome (a, b)	Z cap (both maps)	X cap, beamsplitter	X cap, phase-free
(1, 1) both fire	0.500000	0.000000	0.666667
(1, 0) only a	0.250000	0.500000	0.166667
(0, 1) only b	0.250000	0.500000	0.166667
corr(a, b)	$-1/3$	-1	$-1/5$

The left column is the point. The two weight maps are indistinguishable under the Z cap, because $\lambda(z) = |z|^2$ is the same for both; they induce the same jump chain, the same propensities, the same Gillespie simulation. Under the X cap the beamsplitter phases drive co-firing to exactly zero — the two neurons never both fire, and the two spikes always land on the same soma — while phase-free weights of the same magnitude *enhance* co-firing to $2/3$. The correlation runs from -1 through $-1/3$ to $-1/5$.

Proposition 10.5 (The coherent law is unreachable classically). *No weight map with values in $\mathbb{R}_{\geq 0}$ on the theory of §10.1 produces $\Pr[(1, 1)] = 0$ while leaving $\Pr[(1, 0)], \Pr[(0, 1)] > 0$, under either cap.*

Proof. Under the Z cap the four co-firing derivations of Proposition 10.1 contribute $\sum |t_{1y}t_{2y'}|^2 > 0$ whenever any t is nonzero, and $\Pr[(1, 0)] > 0$ forces some $t_a \neq 0$. Under the X cap the co-firing amplitude is $2(t_{1a}t_{2b} + t_{1b}t_{2a})$, a sum of products of non-negative reals, which vanishes only if $t_{1a}t_{2b} = t_{1b}t_{2a} = 0$; that in turn forces one of $\Pr[(1, 0)]$, $\Pr[(0, 1)]$ to vanish. $\square$

Proposition 10.5 is the honest form of the claim. It is not that this network violates a bound on all classical models — a classical model with correlated routing reproduces the marginals easily — but that within the weighted-GSLT framework, on a fixed theory, the transition from enhanced co-firing to zero co-firing is reachable by the phase and by nothing else.

Remark 10.6 (Bunching, and the m^2 thread). The coherent outcome is that the two spikes always co-locate. This is the bosonic enhancement that the earlier note raised and closed as a normalisation artefact, reappearing where it is not an artefact: Remark 7.6’s m^2 is bunching, and here the derivations it enhances are genuinely distinct — distinguished by injectivity of $c(\ell)$ — and genuinely rejoining. The classical linear-in- m answer and the coherent quadratic-in- m answer are the Z and X columns of the same table.

10.4 Plasticity reads the branch record

In the earlier note plasticity was the weight map’s update function, and its distinguishing feature was that the map may change as the system runs. Under Definition 5.3 that is a different PREPARE state at each step, which §5.4 already handles. What is new here is what the update must *read*.

A Hebbian rule updates $w(\text{relay}, x, y)$, so it must know which synapse carried the spike — both the presynaptic index x and the postsynaptic index y . By Proposition 2.3 that information lives in the branch record, and by Theorem 8.1 reading it is reading a which-path record. We model a partial read: a trace register coupled to the step-one branch bits by a rotation of angle α , so that the two trace states have overlap $c = \cos \alpha$, and then discarded.

The first thing to establish is that the read has to be specific to do damage.

Lemma 10.7 (Single-index reads are free). *For the beamsplitter of §10.3, the four co-firing amplitudes are $-\frac{i}{2}, +\frac{i}{2}, +\frac{i}{2}, -\frac{i}{2}$ under the labelling of Proposition 10.1. Grouping them by the presynaptic index alone, or by the postsynaptic index alone, partitions them into two blocks each containing one amplitude of each sign. Hence each block sums to zero, and decohering either index alone leaves $\Pr[(1, 1)] = 0$ unchanged.*

Proof. Direct computation of the four products, then inspection of the two one-bit partitions. $\square$

So a plasticity rule that records only which neuron fired, or only which release site emptied, costs nothing. A rule that records the synapse — both indices, which is what a synapse-specific update requires — separates all four derivations. Contracting the diagram with both step-one branch bits coupled to the trace register at strength α :

α	$c = \cos \alpha$	$\text{Pr}[\text{both fire}]$	$\text{corr}(a, b)$
0	1.000000	0.000000	-1.000000
$\pi/16$	0.980785	0.000369	-0.999262
$2\pi/16$	0.923880	0.005761	-0.988544
$3\pi/16$	0.831470	0.027618	-0.946248
$4\pi/16$	0.707107	0.079009	-0.853553
$5\pi/16$	0.555570	0.164939	-0.716828
$6\pi/16$	0.382683	0.275929	-0.567486
$7\pi/16$	0.195090	0.393160	-0.435586
$8\pi/16$	0.000000	0.500000	-0.333333

At $c = 0$ the network sits exactly on the Z -cap row of §10.3: 0.500000 and $-1/3$. The curve has a closed form.

Proposition 10.8 (The exchange rate). *Let $\eta = 1 - c$ measure the distinguishability the plasticity read achieves. Then, for the theory and weight map above,*

$$\text{Pr}[\text{both fire}] = \frac{\eta^2}{1 + \eta^2}.$$

Proof. By Proposition 10.1 the co-firing weight is $\sum_{d,d'} A_d \overline{A_{d'}} \langle T_{d'} | T_d \rangle$ over the four derivations, with $A \in \{-\frac{i}{2}, \frac{i}{2}, \frac{i}{2}, -\frac{i}{2}\}$ and trace overlaps c^k where k is the number of indices in which the two derivations differ. The four diagonal terms give 1; the four pairs differing in one index give $-c/2$ each; the two pairs differing in both give $+c^2/2$ each. The total is $1 - 2c + c^2 = (1 - c)^2$. The single-soma outcomes are unaffected — their two derivations differ in the presynaptic index only, and their cross term has vanishing real part — contributing $1/2$ each. Normalising gives the claim. $\square$

The closed form is verified against the contracted diagram at every row of the table to a worst deviation of 1.67×10^{-16} .

Corollary 10.9 (Learning decoheres). *A synapse-specific plasticity rule on this network destroys the coherent correlation at the rate $\eta^2/(1 + \eta^2)$, reaching the classical law exactly at $\eta = 1$. The correlation coefficient runs monotonically from -1 to $-1/3$.*

This is Theorem 8.1 in a network. The branch register is what makes derivations distinguishable, distinguishability is what plasticity needs, and distinguishability is what interference cannot survive. Lemma 10.7 keeps the statement from being vacuous: it is not that observation in general decoheres, but that observation at exactly the granularity a Hebbian update requires does, while coarser observation is free. Whether a useful learning rule exists in the gap — specific enough to update a synapse, coarse enough to leave the interfering derivations unresolved — is the question this example raises and does not answer.

Remark 10.10 (Where the nonlinearity comes from). A standing objection to any quantum network model is that linear maps cannot threshold. Nothing in this section adds a nonlinearity by hand, and none is needed: the McCulloch–Pitts threshold is the join pattern, the join pattern is the demanded tag assignment $\hat{L}_\ell$ of Definition 4.3, and that is a projector — linear on $\mathcal{H}_C$, and a Boolean conjunction on configurations. The Born rule supplies a second, quadratic nonlinearity at the cap. The framework has two sources of nonlinearity before anyone asks for one.

11 Costs and tractable fragments

11.1 Width, depth, ancillas

Proposition 11.1 (Resource count). *The diagram D_β uses*

$$\underbrace{q |\Lambda_\beta(t)|}_{\text{term register}} + \underbrace{\beta \lceil \log_2 N \rceil}_{\text{branch registers}} + \underbrace{\beta \max_j |\text{rd}(j)|}_{\text{match flags}}$$

qubits, where $N = |\mathcal{J}_\beta(t)|$, and has depth $O(\beta \cdot \max_j q |\text{fp}(j)|)$ up to the Toffoli decomposition of the match predicates.

Proof. Immediate from Construction 6.2 and Construction 9.5; flags are reusable across steps when write-detectability holds, giving the stated β -linear rather than β -quadratic count. $\square$

The branch registers are the price of keeping derivations distinct, and by Theorem 8.1 it is not a price that can be avoided by cleverness: the record has to exist in order to be capped. What can be reduced is N . Two observations follow.

11.2 The Clifford fragment

Proposition 11.2 (Clifford weight maps). *Suppose $|w(\gamma, j)|$ is independent of j and $\arg w(\gamma, j) \in \frac{\pi}{2}\mathbb{Z}$ for all γ, j . Then every PREPARE stage is a Clifford operation on B , and the only non-Clifford content of D_β is in the match predicates and the matrix units.*

Proof. Under uniform magnitudes $|\text{prep}(\gamma)\rangle \propto D(\gamma)|+\rangle^{\otimes b}$ with D diagonal with entries in $\{1, i, -1, -i\}$, which is Clifford; $|+\rangle^{\otimes b}$ is Clifford state preparation. $\square$

This gives a design constraint with teeth. Stabiliser decomposition makes contraction of a ZX diagram efficient in the number of non-Clifford phases, so a cwG whose weight phases are multiples of $\pi/2$ is classically simulable at a cost governed by the remaining T -count, while a weight map with generic phases is not. It is a genuine fragment of the calculus, defined by a condition on the weight map rather than on the syntax, and it is where a first implementation should live. The analogy to the ideal-versus-checkable methodology of [1] is exact: the ideal weight codomain is $\mathbb{C}$, and $\frac{\pi}{2}\mathbb{Z}$ -phases with uniform magnitude is a fragment that computes.

11.3 Sparsity

Proposition 3.9 is the practical limit, and its cause is that the encoding is dense: every location in the address set gets q qubits for the whole run, including locations that are $\perp$ at every reachable configuration except a few. Three mitigations, in increasing order of ambition and decreasing order of confidence:

1. *Location pruning.* Construction 6.2 already computes the reachable set; locations that are $\perp$ throughout can be deleted rather than allocated.
2. *Register reuse.* A location that becomes $\perp$ and is never revisited can have its register recycled, provided it is returned to $|0\rangle^{\otimes q}$, which is an uncomputation and is subject to the same detectability question as Proposition 9.7.

3. *Second quantisation.* Where a calculus is presented so that the interesting quantity is an occupancy count at a name rather than a tag at a location — the reading under which the earlier note’s birth–death and spiking-network examples are natural — one may replace $\mathbb{C}^{\Sigma_{\perp}}$ per location by a truncated number state per name, with the truncation bounded by the copy-number and namespace-scope analysis. We flag this as the correct direction for the chemical-reaction-network style of example and do not develop it here.

12 Implementation

12.1 Emission

The compiler’s output is a ZX diagram, and there are two reasonable serialisations.

1. *Direct PyZX graph.* Emit vertices with type (Z , X , boundary, Hadamard), phase as a fraction of π , and edges with type (simple, Hadamard), plus a global scalar. This is the shortest path to `full_reduce`, `extract_circuit`, and the tensor-based equality check for small instances. The PyZX API and its graph JSON should be pinned by version; we do not rely here on any specific serialisation detail.
2. *A monoidal intermediate.* Emit into a general symmetric-monoidal term representation with ZX as one interpretation, and generate PyZX from that. This costs a layer and buys the option of retargeting to a calculus with a primitive for linear combination, which would let §5.5 drop the branch register in favour of a native sum. Given Theorem 8.1 that trade is less attractive than it first appears — the branch register is not merely an encoding device, it is where the semantics is chosen — but the option has independent value for the qudit question of Remark 3.7.

We recommend (1) for the first implementation and (2) only if the qudit or linear-combination questions become pressing.

12.2 The pipeline

1. cwG program + budget $\longrightarrow$ Construction 6.2 $\longrightarrow$ address set, jump set, footprints.
2. $\longrightarrow$ ZX diagram D_{β} with the branch wires open.
3. Choose a cap (Table 1).
4. Simplify. Verify against the reference by contraction for small instances.
5. Attempt extraction; on success, a circuit; on failure, a measurement pattern.

12.3 What changes in the existing crate

The weighted simulator `rho-weighted` is a simulator, not an interpreter, and it owns its own state; the compiler described here is a third artefact with the same relationship to both. It shares the parts of the simulator that are about the calculus and shares none of the parts that are about sampling. Concretely:

- *Reused as-is.* The `Formula` AST and its structural fragment; the R3 machinery (`is_structural`, `Checkable::try_key`, `Partition::from_checked_keys`) — which §5.4 shows to be a prerequisite rather than a hygiene measure; the partition check; the exhaustive graph construction, which becomes the reference oracle for the differential tests below.
- *Generalised.* `RateValue::complex` loses its interpretation as an amplitude of a jump operator and becomes an amplitude of a PREPARE state; `rate()` returning $|z|^2$ remains correct but is now specifically the Z -cap reading (Remark 2.8).
- *New.* The address-set computation of Construction 6.2; the footprint calculation of Definition 4.1; the qubit layout; the ZX emitter.
- *Not reused.* `QctmcModel` and the Lindblad/quantum-jump apparatus. The $\sqrt{\Sigma}$ aggregation that `from_graph_unchecked` performs is a *classical* lift, and by Remark 7.6 it is the right thing for the Z cap and the wrong thing for the X cap. It should stay where it is, serving the simulator, and should not be carried into the compiler.

12.4 Test obligations

1. *Semigroup.* $D_{\beta+1} = (\text{id} \otimes V) \circ D_{\beta}$ by ZX-rewriting (Proposition 5.10).
2. *Classical conservativity.* For a cwG with all weights real and non-negative, the Z -capped diagram's jump distribution must agree with `rho-weighted`'s propensities on the same theory, to numerical tolerance. This is Theorem 7.1 as a differential test against an independently written engine, and it is the one test that would catch an error in the address-set or footprint computation.
3. *Multiplicity.* A fixture with m distinct locations firing the same rule must show Z -cap probability linear in m and X -cap weight quadratic in m (Remark 7.6), with a negative control asserting that the two are distinguishable — the analogue of the existing negative control `sum_of_roots_would_have_given_m_squared`.
4. *Which-path.* A fixture with a closed diamond must show interference under the X cap and none under the Z cap, and the retained- B reduced state must equal the Z -capped mixture (Theorem 8.1(i)).
5. *Dilation.* For a uniformised weight map, $V^{\dagger}V = \text{id}$ to tolerance; for a non-uniformised one, the test must fail, as a control.
6. *The worked example.* The three tables of §10.3 and §10.4 are regression fixtures: the Z -cap law must be invariant under replacing the beamsplitter by its phase-free counterpart, the X -cap co-firing probability must be zero to numerical tolerance, and the plasticity curve must match Proposition 10.8.
7. *Structural invariance.* Two runs of Construction 6.2 differing only in the order in which jumps are enumerated must emit ZX-equal diagrams. Note that this is *not* a confluence assumption: it is invariance of the emitted diagram under the compiler's own scheduling, which holds because the sum in Definition 5.8 is over all jumps regardless of order, and which would be false for a compiler that committed to a derivation.

13 Open threads

Obligation 13.1 (Tuple-*receive* elaboration). Both source papers treat the elaboration of a MeTTaIL tuple-pattern *receive* into nested single-name *receives* with name-equality guards as unnecessary detail. This note works at the granularity of the context-labelled transition system, where one firing is one labelled transition, and is therefore insulated from the question. But a compiler that targeted core rho directly would not be: a partially-consumed failing match is a consume-and-restore loop whose intermediate steps can interleave, and interleaving administrative steps that rejoin form closed diamonds, which by Corollary 7.5 *interfere*. Spurious interference from an encoding artefact would be invisible in a trace and would corrupt every amplitude. The obligation is to show that the elaboration introduces no rejoining administrative branching, or to confine the compiler to the CLTS granularity by fiat and say so.

Obligation 13.2 (The geometric fragment versus the Boolean demand). The classifying-topos apparatus of [2, §4.4] sends a bisimulation-preserving translation to a *geometric* morphism, and geometric morphisms preserve only the geometric fragment: finite limits and arbitrary colimits one way, limits the other. But the partition discipline of Convention 2.6 requires a complement — the default class is the complement of the union of the keys — and [1] proves independently that weighting cannot be specified over a target logic with finite limits only. So the weight map’s keys cannot in general be transported along the desugaring by the internal-logic machinery.

This note sidesteps the difficulty rather than solving it: the key is resolved at compile time, on the source side, where the Boolean structure exists, and its identity is carried in the jump index $j = (r, \ell, \kappa)$ and hence in a basis vector of B . Classification never has to be performed in the target’s internal logic. Whether the sidestep can be upgraded — whether there is a Boolean-preserving transport, or a characterisation of the keys that do transport geometrically — is open, and it is the piece of theory this construction most obviously wants.

Obligation 13.3 (Channel-naming optimality versus weight-class separation). The optimal channel-naming scheme of [3] deliberately assigns one channel to contexts that the firing semantics cannot distinguish; the desugaring used here assigns one channel per location and is, in that paper’s terms, the “reflect the context verbatim” choice, which fails symbol-once optimality. The two induce the same context-labelled transition system [2, Rem. 4.6], so this note is indifferent. A compiler that wanted the optimality would not be: collapsing two contexts collapses two jump indices, hence two basis vectors of B , hence two derivations that Theorem 7.4 would otherwise keep apart. The repair is available and is a small result in its own right: build the set automaton from $\text{LHS} \cup \{\text{keys}\}$ rather than from LHS alone — legitimate exactly because Convention 2.7 makes keys structural, hence patterns — so that the coarsest-sound partition is automatically fine enough to separate weight classes. Proving that the augmented automaton retains the three optimality theorems is the work.

Obligation 13.4 (Partial caps and decoherence). Corollary 8.2 notes that measuring some qubits of B and post-selecting others decoheres a chosen quotient of the derivations. This is the natural formal home for a notion of observation-induced decoherence — a shard that is watched loses coherence in exactly the derivations the watcher can distinguish — and it is not developed here.

Obligation 13.5 (Qudits). Definition 3.4 is qudit-native and §5.1 compiles it away because the tooling is qubit-native. A qudit ZX variant would remove the encoding step, shrink the

match predicates, and make Lemma 5.2 a one-line statement about $\Sigma_{\perp}$ -spiders. The question is whether the tooling justifies the theory.

Obligation 13.6 (Second quantisation). §11.3 sketches a reading in which occupancy at a name replaces tag-at-a-location. For the chemical-reaction-network and spiking-network examples of [1] that reading is the natural one, and it connects the width bound directly to the copy-number and namespace-scope analysis. Whether the two encodings agree on a common fragment, and what the translation between them is, is open.

14 Conclusion

The quantum reading of a weighted GSLT went wrong by trying to be the stochastic reading over a larger field. Compiling to the ZX-calculus fixes it not by finding a better normalisation but by moving to a target where amplitude and phase have separate places to live: the magnitude of a weight is what the Z cap sees and its argument is what the X cap sees, and the two readings are the same diagram terminated differently.

Three things made this available and none of them was invented here. The desugaring of every GSLT into rho by location-indexed installation supplies an address space and, because its location channels are injective, supplies derivation identities — which is exactly the codomain indexed by derivations that the earlier note deferred. The requirement that keys be structural, arrived at under referee pressure for reasons of locality, turns out to be the condition under which a rewrite has a footprint at all and the condition under which key formulae compile to bounded-arity gadgets. The fictitious-jump convention, adopted so that a generator’s diagonal and a simulator’s total propensity would agree, turns out to be what makes a normal form a fixed point and hence what makes uniformisation — and with it extractability — possible.

The worked example says what the phase is for. Two weight maps that a Gillespie simulator cannot tell apart — same propensities, same jump chain, same everything the classical theory can see — differ, coherently, between neurons that never fire together and neurons that fire together two times in three. And the plasticity result is the framework criticising itself: the branch record is what makes derivations distinguishable, a Hebbian update needs exactly that distinguishability, and interference cannot survive it.

What is genuinely new is small and, we think, correct: that the sum over enabled redexes must be carried by an explicit branch register; that the weight map is best understood as the amplitudes of a state on that register; that a retained branch record is a which-path record, so that interference between derivations can be effected by projection and by nothing else; and hence that measurement is not the concession one makes for programs that fail to be unitary but the mechanism by which the quantum reading differs from the classical one at all.

References

- [1] L. G. Meredith. *Weighted Graph-Structured Lambda Theories: Stochastic and quantum execution for MeTTaIL, with a spiking recurrent network as a worked example*. Second version, F1R3FLY.io, 2026.
- [2] L. G. Meredith. *Knotted Topoi*. F1R3FLY.io, 2026. (Desugaring of MeTTaIL into rho, §4.2 and Appendix A; operational correspondence, Prop. 4.5; OSLF as internal logic, §4.4.)

- [3] L. G. Meredith. *Optimal Channel Naming for Compositional Rewrite Translations via Set Automaton Partial Evaluation*. F1R3FLY.io, 2026.
- [4] L. G. Meredith. *Graph-Structured Lambda Theories: A Reference for the Working Developer*. F1R3FLY.io, 2026.
- [5] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electr. Notes Theor. Comput. Sci.*, 141(5):49–67, 2005.
- [6] M. Bouwman and R. Erkens. Term rewriting based on set automaton matching. 2022.
- [7] B. Coecke and A. Kissinger. *Picturing Quantum Processes*. Cambridge University Press, 2017.
- [8] V. Danos, E. Kashefi, and P. Panangaden. The measurement calculus. *J. ACM*, 54(2), 2007.
- [9] A. Kissinger and J. van de Wetering. PyZX: Large scale automated diagrammatic reasoning. *EPTCS* 318, 2020.
- [10] D. T. Gillespie. Exact stochastic simulation of coupled chemical reactions. *J. Phys. Chem.*, 81(25):2340–2361, 1977.